

Docker Swarm

Cluster Haute Disponibilité

Continuité et Reprise d'Activité (PCA/PRA)

1. Introduction

Ce document retrace l'ensemble des procédures techniques réalisées dans le cadre du déploiement d'un cluster Docker Swarm. Il est destiné à servir de référence pour la compréhension de l'infrastructure mise en place, et à documenter les tests de continuité et de reprise d'activité (PCA/PRA).

1.1 Qu'est-ce que Docker ?

Docker est une plateforme de conteneurisation qui permet d'empaqueter une application et toutes ses dépendances dans une unité standardisée appelée conteneur. Contrairement à une machine virtuelle, un conteneur ne virtualise pas un système d'exploitation complet : il partage le noyau de la machine hôte, ce qui le rend beaucoup plus léger et rapide à démarrer.

1.2 Qu'est-ce que Docker Swarm ?

Docker Swarm est l'outil d'orchestration natif de Docker. Il permet de regrouper plusieurs machines (appelées nœuds) en un cluster, et de gérer automatiquement le déploiement, la répartition et la reprise des conteneurs sur l'ensemble de ces machines.

En cas de panne d'un nœud, Swarm détecte la défaillance et redémarre automatiquement les conteneurs affectés sur les nœuds disponibles. C'est ce mécanisme qui garantit la haute disponibilité des services.

1.3 Objectifs du projet

- Déployer un cluster Swarm sur 5 VMs Debian sous VMware
- Assurer la haute disponibilité et la redondance des services
- Mettre en place un stockage persistant partagé via NFS
- Déployer les services applicatifs critiques
- Tester et documenter les procédures PCA/PRA

2. Architecture du cluster

Le cluster repose sur 5 machines virtuelles Debian 12 déployées sous VMware Workstation. Chaque VM dispose de deux interfaces réseau :

- Carte NAT (ens32) : accès à internet pour télécharger les paquets
- Carte Host-Only VMnet1 (ens34) : réseau privé entre les VMs (192.168.56.0/24)

2.1 Tableau des nœuds

Rôle	Hostname	Adresse IP	RAM	CPU	Disque
Manager	swarm-manager	192.168.56.10	2 Go	2 vCPU	15 Go
Worker 1	swarm-worker1	192.168.56.11	2 Go	2 vCPU	15 Go
Worker 2	swarm-worker2	192.168.56.12	2 Go	2 vCPU	15 Go
Worker 3	swarm-worker3	192.168.56.13	2 Go	2 vCPU	15 Go
NFS	swarm-nfs	192.168.56.20	512 Mo	1 vCPU	20 Go

2.2 Services déployés

Service	Rôle
Registry	Dépôt d'images Docker local sécurisé
MariaDB	Base de données en haute disponibilité
PHP	Environnement d'exécution applicatif
Nginx	Proxy inverse et serveur web
VSCoDe Server	Environnement de développement collaboratif

3. Préparation des machines virtuelles

3.1 Import de l'image de base

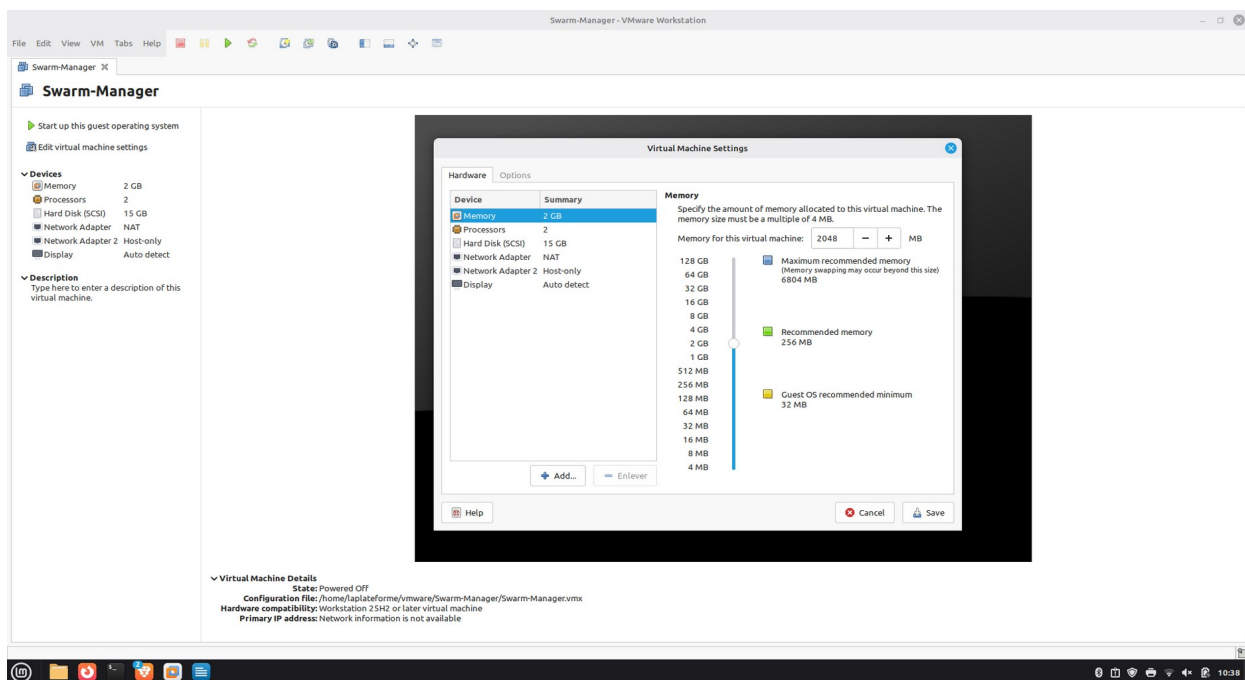
L'infrastructure repose sur une image Debian 12 (.ova) fournie comme base. Cette image a été importée dans VMware Workstation, puis clonée 4 fois pour obtenir les 5 nœuds du cluster.

Procédure d'import

- Ouvrir VMware Workstation
- Fichier → Ouvrir → sélectionner le fichier .ova
- Nommer la VM swarm-manager et valider l'import

Configuration des ressources (swarm-manager et workers)

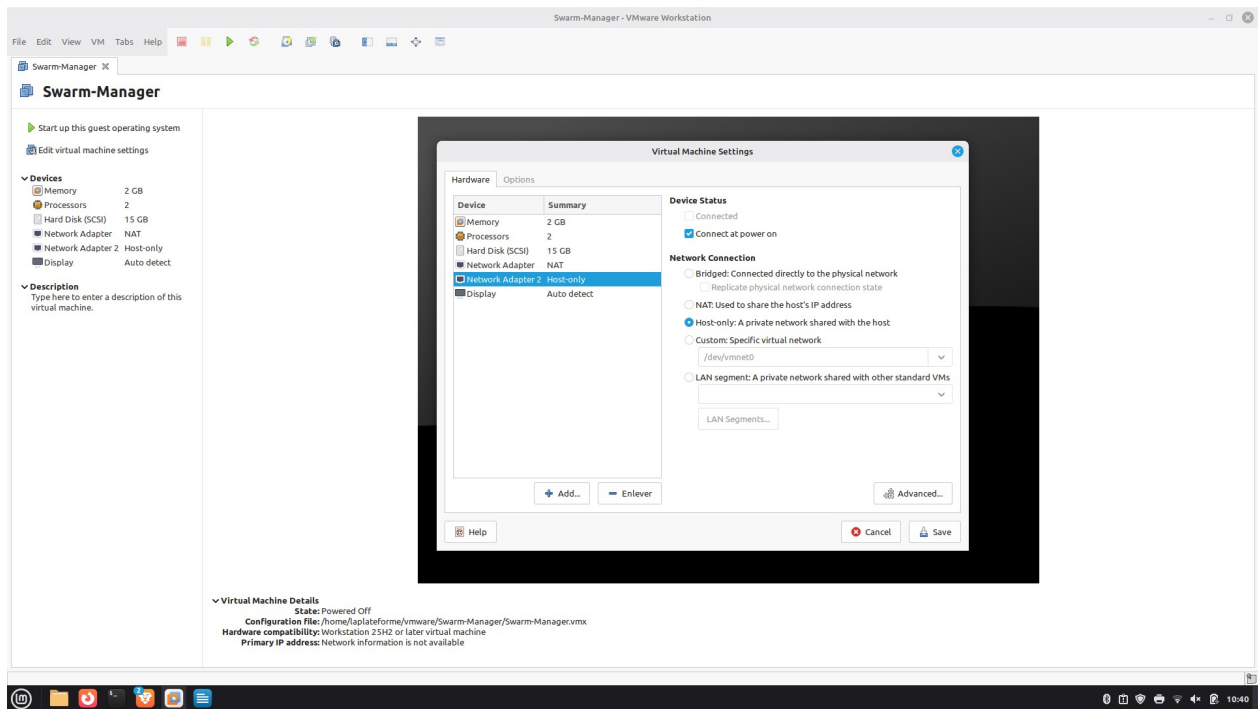
Paramètre	Valeur
Memory	2048 Mo
Processors	2 vCPU
Hard Disk	15 Go



3.2 Ajout de la carte réseau Host-Only

Avant de cloner, une deuxième carte réseau est ajoutée sur swarm-manager afin que tous les clones en héritent automatiquement.

- VM éteinte → VM → Paramètres → Ajouter → Adaptateur réseau
- Sélectionner Host-only (VMnet1)



3.3 Clonage des VMs

Depuis swarm-manager, 4 clones complets sont créés via : clic droit → Gérer → Cloner → Clone complet.

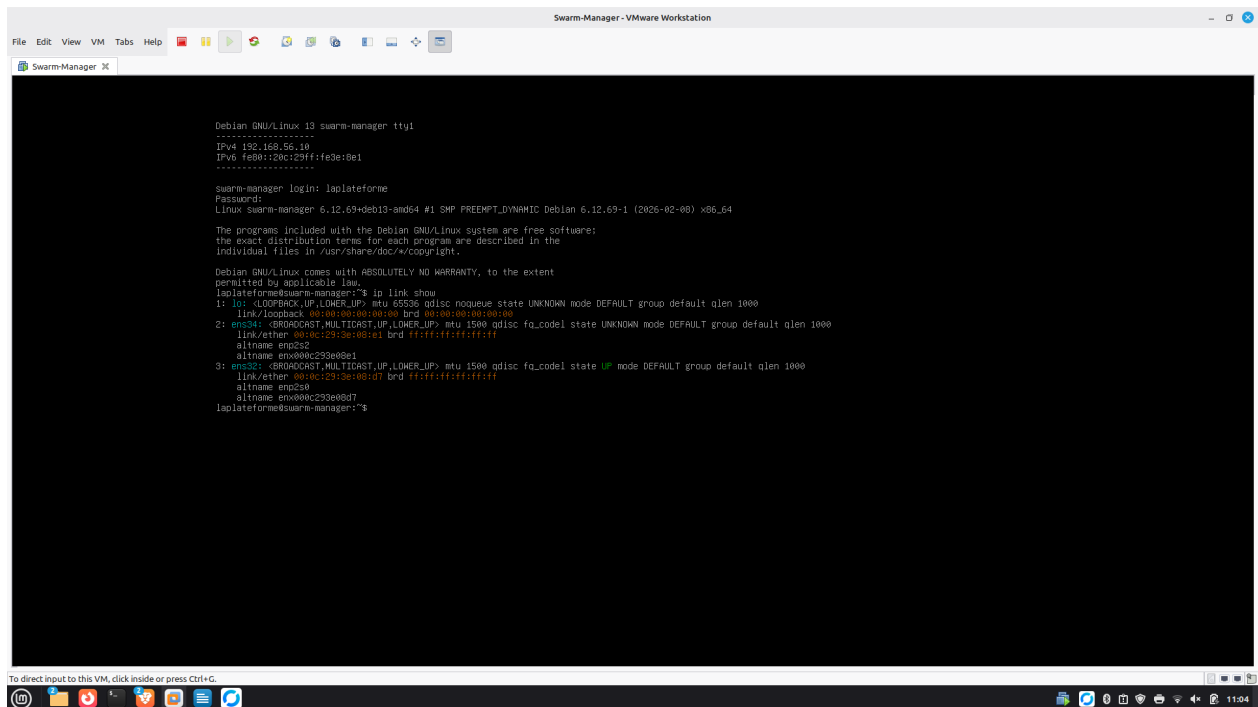
Clone	Hostname	IP
Clone 1	swarm-worker1	192.168.56.11
Clone 2	swarm-worker2	192.168.56.12
Clone 3	swarm-worker3	192.168.56.13
Clone 4	swarm-nfs	192.168.56.20

4. Configuration réseau des nœuds

Après le clonage, chaque VM est configurée individuellement. Les étapes sont identiques sur tous les nœuds, seuls le hostname et l'adresse IP changent.

4.1 Vérification des interfaces réseau

La commande suivante permet d'identifier les noms des cartes réseau : **ip link show**
 Résultat attendu : deux interfaces (hors lo) — ens32 (NAT) et ens34 (Host-Only).



4.2 Configuration de l'IP fixe

L'adresse IP fixe est configurée sur la carte Host-Only (ens34) en éditant le fichier `/etc/network/interfaces` :

```

sudo nano /etc/network/interfaces

# Ajouter à la fin du fichier :
auto ens34
iface ens34 inet static
    address 192.168.56.10    # adapter selon la VM
    netmask 255.255.255.0

```

Appliquer la configuration :

```

sudo systemctl restart networking

```

Vérifier que l'IP est bien attribuée :

```

ip addr show ens34

```

```

laplateforme@swarm-manager:~$ ip addr show ens34
2: ens34: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UNKNOWN group default qlen 1000
    link/ether 00:0c:29:3e:08:e1 brd ff:ff:ff:ff:ff:ff
    altname enp2s2
    altname enx000c293e08e1
    inet 192.168.56.10/24 brd 192.168.56.255 scope global ens34
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe3e:8e1/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever

```

4.3 Remplissage de `/etc/hosts`

Pour que les VMs se trouvent par leur nom, on renseigne le fichier `/etc/hosts` sur chaque nœud :

```

sudo nano /etc/hosts

```

```
# Ajouter ces lignes :
192.168.56.10  swarm-manager
192.168.56.11  swarm-worker1
192.168.56.12  swarm-worker2
192.168.56.13  swarm-worker3
192.168.56.20  swarm-nfs
```

4.4 Changement du hostname

```
sudo hostnamectl set-hostname swarm-manager # adapter selon la VM
```

4.5 Régénération des clés SSH

Comme toutes les VMs sont des clones, elles partagent initialement les mêmes clés SSH. Pour des raisons de sécurité, on régénère des clés uniques sur chaque nœud :`sudo rm /etc/ssh/ssh_host_*`

```
sudo dpkg-reconfigure openssh-server
sudo systemctl restart ssh
```

4.6 Test de connectivité

Depuis swarm-manager, vérifier que toutes les VMs se joignent :

```
ping -c 2 swarm-worker1
ping -c 2 swarm-worker2
ping -c 2 swarm-worker3
ping -c 2 swarm-nfs
```

```
laplateforme@swarm-manager:~$ ping -c 2 swarm-worker1
PING swarm-worker1 (192.168.56.11) 56(84) bytes of data.
64 bytes from swarm-worker1 (192.168.56.11): icmp_seq=1 ttl=64 time=3.26 ms
64 bytes from swarm-worker1 (192.168.56.11): icmp_seq=2 ttl=64 time=1.89 ms

--- swarm-worker1 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 1.892/2.573/3.255/0.681 ms
laplateforme@swarm-manager:~$ ping -c 2 swarm-worker2
PING swarm-worker2 (192.168.56.12) 56(84) bytes of data.
64 bytes from swarm-worker2 (192.168.56.12): icmp_seq=1 ttl=64 time=4.85 ms
64 bytes from swarm-worker2 (192.168.56.12): icmp_seq=2 ttl=64 time=1.44 ms

--- swarm-worker2 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 1.440/3.144/4.848/1.704 ms
laplateforme@swarm-manager:~$ ping -c 2 swarm-worker3
PING swarm-worker3 (192.168.56.13) 56(84) bytes of data.
64 bytes from swarm-worker3 (192.168.56.13): icmp_seq=1 ttl=64 time=3.74 ms
64 bytes from swarm-worker3 (192.168.56.13): icmp_seq=2 ttl=64 time=1.47 ms

--- swarm-worker3 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 1.471/2.606/3.742/1.135 ms
laplateforme@swarm-manager:~$ ping -c 2 swarm-nfs
PING swarm-nfs (192.168.56.20) 56(84) bytes of data.
64 bytes from swarm-nfs (192.168.56.20): icmp_seq=1 ttl=64 time=4.57 ms
64 bytes from swarm-nfs (192.168.56.20): icmp_seq=2 ttl=64 time=1.52 ms

--- swarm-nfs ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 1.520/3.044/4.568/1.524 ms
```

5. Installation de Docker

Docker est installé sur les 4 nœuds Swarm (manager + workers). Le serveur NFS n'en a pas besoin car il ne fait que du stockage.

5.1 Procédure d'installation

```
# 1. Installer les dépendances
sudo apt install -y ca-certificates curl gnupg

# 2. Ajouter la clé GPG officielle Docker
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/debian/gpg | \
  sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

# 3. Ajouter le dépôt Docker
echo "deb [arch=$(dpkg --print-architecture) \
  signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/debian \
  $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# 4. Installer Docker
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin

# 5. Ajouter l'utilisateur au groupe docker
```

```
sudo usermod -aG docker $USER
newgrp docker
```

5.2 Correction IPv6

Par défaut, Docker tente de joindre Docker Hub en IPv6, ce qui échoue si IPv6 n'est pas configuré sur les VMs. On force l'utilisation d'IPv4 en créant le fichier de configuration du daemon :

```
sudo nano /etc/docker/daemon.json

# Contenu du fichier (JSON strict, sans commentaires) :
{
  "ipv6": false
}

sudo systemctl restart docker
```

5.3 Vérification

```
docker --version
docker run hello-world
```

```
laplateforme@swarm-worker2:~$ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:f9078146db2e05e794366b1bfe584a14ea6317f44027d10ef7dad65279026885
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
```


6. Initialisation du cluster Swarm

Le cluster Swarm est initialisé depuis le nœud manager. Cette commande désigne swarm-manager comme chef d'orchestre du cluster et génère un token unique que les workers utiliseront pour rejoindre le Swarm.

NB : Le Swarm fonctionne sur le port 2377 . Les workers communiquent avec le manager via l'IP Host-Only (192.168.56.10), pas via NAT.

6.1 Initialisation du Manager

```
docker swarm init --advertise-addr 192.168.56.10
```

La commande affiche un token à copier soigneusement – il sera utilisé sur chaque worker.

```
laplateforme@swarm-manager:~$ docker swarm init --advertise-addr 192.168.56.10
Swarm initialized: current node (7jz128rb59ylxvzmuzao54p9w) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-3qbvwfae1sm30xcgn4hkjq0lure71fn3ckzpz3nsq1hr0yyru7-2o443t98hv5jb8u1g8z45j6sx 192.168.56.10:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.
```

6.2 Ajout des Workers

Sur swarm-worker1, swarm-worker2 et swarm-worker3, exécuter la commande affichée par le manager :

```
docker swarm join --token SWMTKN-1-xxx... 192.168.56.10:2377
```

```
laplateforme@swarm-worker1:~$ docker swarm join --token SWMTKN-1-3qbvwfae1sm30xcgn4hkjq0lure71fn3ckzpz3nsq1hr0yyru7-2o443t98hv5jb8u1g8z45j6sx 192.168.56.10:2377
This node joined a swarm as a worker.
```

6.3 Vérification du cluster

Sur le manager, vérifier que les 4 nœuds sont bien présents et actifs :

```
docker node ls
```

```
laplateforme@swarm-manager:~$ docker node ls
```

ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER STATUS	ENGINE VERSION
7jz128rb59ylxvzmuzao54p9w *	swarm-manager	Ready	Active	Leader	29.4.2
a05ee633fkd1zopqdkz0fgm83	swarm-worker1	Ready	Active		29.4.2
zqjfok95ed5gpub6oygs18r7	swarm-worker2	Ready	Active		29.4.2
z8t8m83r1yc7npkjac09wx0yq	swarm-worker3	Ready	Active		29.4.2

7. Configuration du serveur NFS

Le NFS (Network File System) permet à tous les workers de partager le même espace de stockage. C'est essentiel pour la persistance des données : si Swarm déplace un conteneur d'un worker à un autre, les données restent accessibles car elles sont stockées sur le serveur NFS centralisé.

7.1 Installation du serveur NFS (sur swarm-nfs)

```
sudo apt update && sudo apt install -y nfs-kernel-server

# Créer le dossier partagé
sudo mkdir -p /srv/nfs/swarm
sudo chown nobody:nogroup /srv/nfs/swarm
sudo chmod 777 /srv/nfs/swarm
```

7.2 Configuration du partage

```
sudo nano /etc/exports

# Ajouter cette ligne :
/srv/nfs/swarm 192.168.56.0/24(rw, sync, no_subtree_check, no_root_squash)
```

Appliquer la configuration :

```
sudo exportfs -a
sudo systemctl restart nfs-kernel-server
sudo systemctl status nfs-kernel-server
```

```
laplateforme@swarm-nfs:~$ sudo systemctl status nfs-kernel-server
● nfs-server.service - NFS server and services
   Loaded: loaded (/usr/lib/systemd/system/nfs-server.service; enabled; preset: enabled)
   Active: active (exited) since Mon 2026-05-04 14:52:44 CEST; 9s ago
  Invocation: 6d6b93f076374449948c12ebb89e5635
     Docs: man:rpc.nfsd(8)
           man:exportfs(8)
   Process: 12361 ExecStartPre=/usr/sbin/exportfs -r (code=exited, status=0/SUCCESS)
   Process: 12362 ExecStart=/bin/sh -c /usr/sbin/nfsdctl autostart || /usr/sbin/rpc.nfsd (code=exited, status=0/SUCCESS)
  Main PID: 12362 (code=exited, status=0/SUCCESS)
    Mem peak: 1.8M
       CPU: 28ms

mai 04 14:52:44 swarm-nfs systemd[1]: Starting nfs-server.service - NFS server and services...
mai 04 14:52:44 swarm-nfs sh[12364]: nfsdctl: lockd configuration failure
mai 04 14:52:44 swarm-nfs systemd[1]: Finished nfs-server.service - NFS server and services.
```

7.3 Montage NFS sur les workers

Sur swarm-worker1, worker2 et worker3 :

```
# Installer le client NFS
sudo apt install -y nfs-common

# Créer le point de montage
sudo mkdir -p /mnt/nfs/swarm

# Monter le partage
sudo mount 192.168.56.20:/srv/nfs/swarm /mnt/nfs/swarm
```

```
# Vérifier le montage
df -h | grep nfs
```

7.4 Montage permanent (fstab)

Pour que le montage NFS survive aux redémarrages, ajouter une entrée dans `/etc/fstab` sur chaque worker :

```
sudo nano /etc/fstab
```

```
# Ajouter à la fin :
192.168.56.20:/srv/nfs/swarm /mnt/nfs/swarm nfs defaults 0 0
```

```
laplateforme@swarm-worker1:~$ df -h | grep nfs
192.168.56.20:/srv/nfs/swarm 7,4G 2,1G 4,9G 31% /mnt/nfs/swarm
```

8. Déploiement des services

Les services sont déployés via des fichiers YAML appelés stacks. Chaque stack décrit un ou plusieurs services : l'image Docker à utiliser, les ports exposés, les volumes, le nombre de réplicas et les contraintes de placement.

NB : Dans Docker Swarm, on utilise 'docker stack deploy' à la place de 'docker-compose up' . Le principe est identique mais Swarm répartit automatiquement les conteneurs sur les workers.

8.0 Prérequis — Montage NFS sur le manager

Le manager doit aussi avoir accès au NFS car certains services (comme le Registry) y sont contraints. Sans ce montage, les conteneurs sont rejetés.

```
sudo apt install -y nfs-common
sudo mkdir -p /mnt/nfs/swarm
sudo mount 192.168.56.20:/srv/nfs/swarm /mnt/nfs/swarm

# Rendre permanent :
sudo nano /etc/fstab
# 192.168.56.20:/srv/nfs/swarm /mnt/nfs/swarm nfs defaults 0 0
```

8.1 Organisation des fichiers

Les fichiers de stacks sont créés dans un dossier dédié sur le manager :

```
mkdir -p ~/stacks
cd ~/stacks
```

8.2 Registry — Dépôt d'images local\$

Le Registry est le dépôt d'images Docker interne du cluster. Il permet de stocker et distribuer les images sans dépendre de Docker Hub.

```
# Créer le dossier sur le NFS (depuis swarm-nfs) :
sudo mkdir -p /srv/nfs/swarm/registry
sudo chmod 777 /srv/nfs/swarm/registry
# registry.yml
version: '3.8'
```

```

services:
  registry:
    image: registry:2
    ports:
      - "5000:5000"
    volumes:
      - /mnt/nfs/swarm/registry:/var/lib/registry
    deploy:
      replicas: 1
      placement:
        constraints:
          - node.role == manager

docker stack deploy -c registry.yml registry

```

8.3 MariaDB — Base de données

MariaDB est déployé sur un worker avec ses données stockées sur le NFS pour garantir la persistance même en cas de redémarrage ou migration du conteneur.

```

# Créer le dossier sur le NFS :
sudo mkdir -p /srv/nfs/swarm/mariadb

# mariadb.yml
version: '3.8'

services:
  mariadb:
    image: mariadb:10.11
    environment:
      MYSQL_ROOT_PASSWORD: rootpassword
      MYSQL_DATABASE: swarmdb
      MYSQL_USER: swarmuser
      MYSQL_PASSWORD: swarmpassword
    volumes:
      - /mnt/nfs/swarm/mariadb:/var/lib/mysql
    deploy:
      replicas: 1
      placement:
        constraints:
          - node.role == worker

docker stack deploy -c mariadb.yml mariadb

```

8.4 Nginx + PHP — Serveur web

Nginx et PHP sont déployés ensemble dans une même stack avec 2 réplicas chacun pour assurer la haute disponibilité. Nginx reçoit les requêtes HTTP et les transmet à PHP-FPM pour l'exécution des scripts.

```

# Créer les dossiers sur le NFS :
sudo mkdir -p /srv/nfs/swarm/html
sudo mkdir -p /srv/nfs/swarm/nginx
sudo chmod 777 /srv/nfs/swarm/html

# Créer une page de test :
echo '<?php phpinfo(); ?>' | sudo tee /srv/nfs/swarm/html/index.php

```

Configuration Nginx à créer sur le NFS :

```
sudo nano /srv/nfs/swarm/nginx/default.conf
```

```
server {
    listen 80;
    root /var/www/html;
    index index.php index.html;

    location / {
        try_files / =404;
    }

    location ~ .php$ {
        fastcgi_pass php:9000;
        fastcgi_index index.php;
        include fastcgi_params;
        fastcgi_param SCRIPT_FILENAME ;
    }
}
```

```
# web.yml
version: '3.8'
```

```
services:
  php:
    image: php:8.2-fpm
    volumes:
      - /mnt/nfs/swarm/html:/var/www/html
    deploy:
      replicas: 2
      placement:
        constraints:
          - node.role == worker

  nginx:
    image: nginx:alpine
    ports:
      - "80:80"
    volumes:
      - /mnt/nfs/swarm/html:/var/www/html
      - /mnt/nfs/swarm/nginx:/etc/nginx/conf.d
    deploy:
      replicas: 2
      placement:
        constraints:
          - node.role == worker
```

```
docker stack deploy -c web.yml web
```

```
laplateforme@swarm-manager:~/stacks$ docker service ls
ID                NAME                MODE                REPLICAS    IMAGE                PORTS
cx1wyfzzqnxr     mariadb_mariadb     replicated          1/1          mariadb:10.11
28ok7v5226lg     registry_registry   replicated          1/1          registry:2           *:5000->5000/tcp
ktbe7yqe6rkq     vscode_vscode        replicated          1/1          codercom/code-server:latest *:8080->8080/tcp
spbrkoohowen     web_nginx           replicated          2/2          nginx:alpine         *:80->80/tcp
2vxj0y9c2rx4     web_php             replicated          2/2          php:8.2-fpm
```

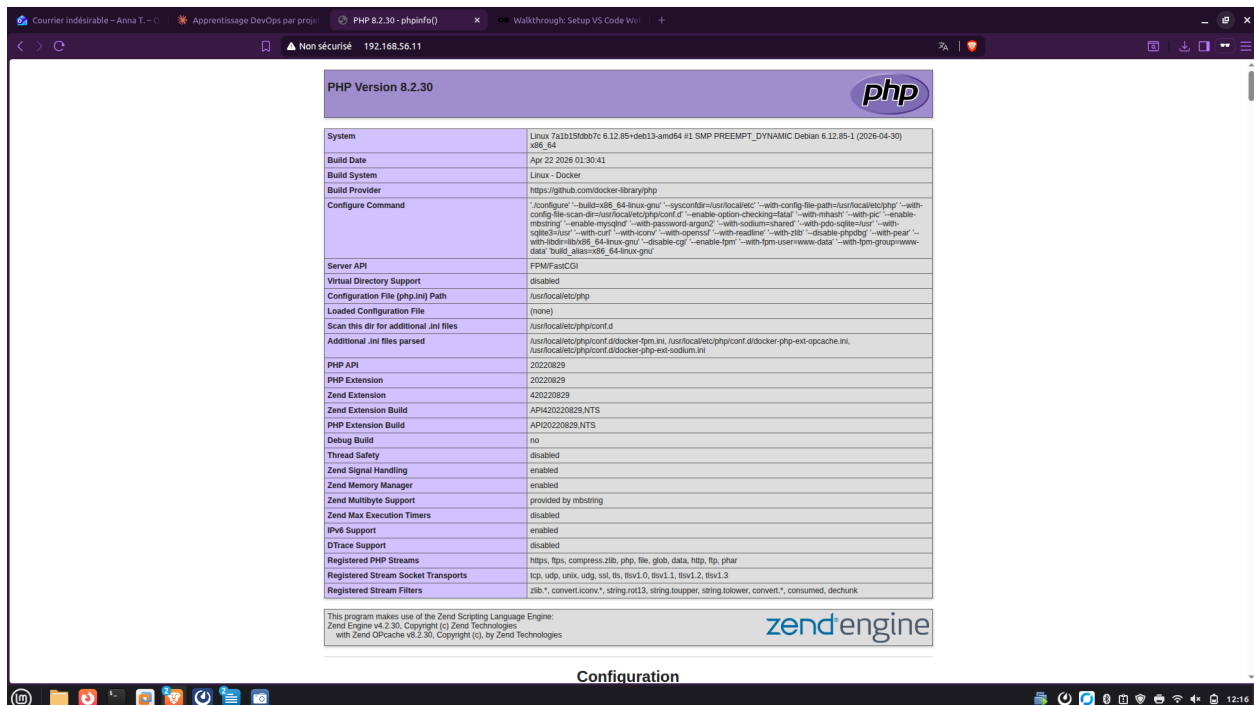
8.5 Accès depuis la machine hôte

Par défaut, la machine hôte Linux n'a pas d'IP sur le réseau VMnet1 (192.168.56.0/24). Il faut lui en assigner une pour accéder aux services depuis le navigateur.

```
# Vérifier l'interface VMnet1 :
ip link show | grep vmnet

# Assigner une IP temporaire :
sudo ip addr add 192.168.56.1/24 dev vmnet1

# Rendre permanent (dans /etc/network/interfaces) :
auto vmnet1
iface vmnet1 inet static
    address 192.168.56.1
    netmask 255.255.255.0
```



8.6 VSCode Server — Environnement de développement

VSCode Server permet d'accéder à un environnement de développement complet depuis le navigateur, sans installation locale.

```
# vscode.yml
version: '3.8'

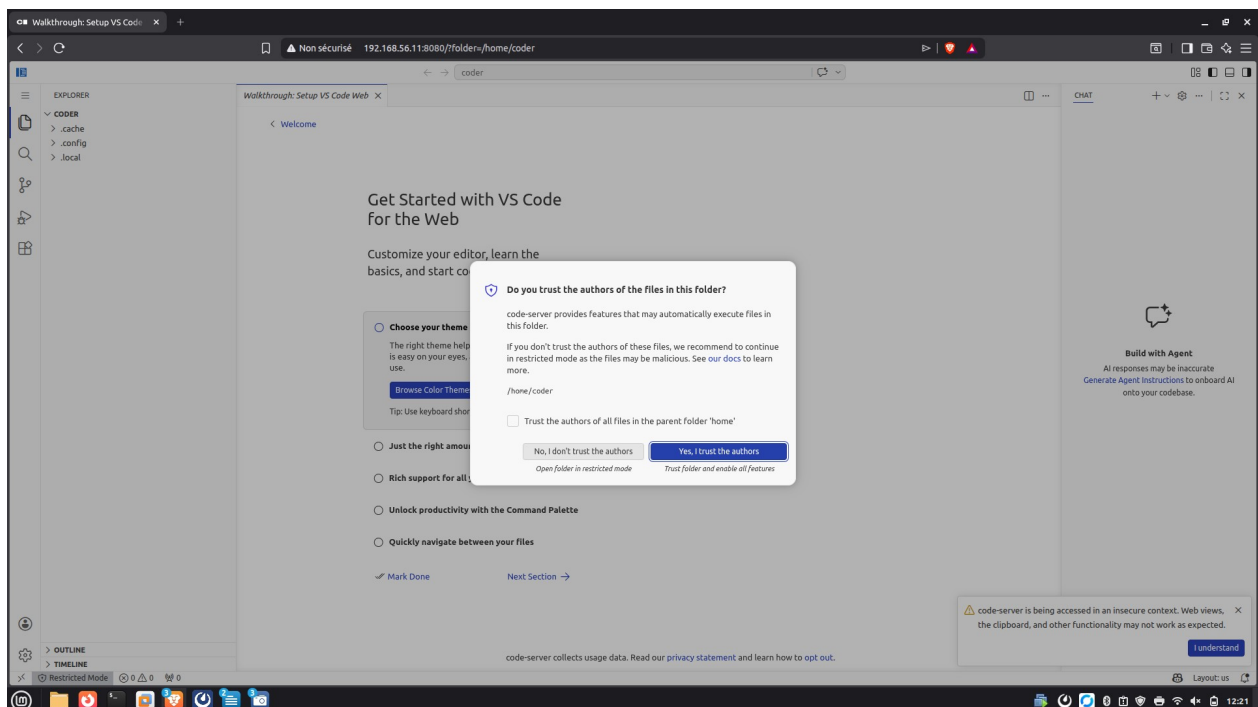
services:
  vscode:
    image: codercom/code-server:latest
    ports:
```

```

- "8080:8080"
volumes:
- /mnt/nfs/swarm/vscode:/home/coder
environment:
- PASSWORD=vscode123
deploy:
  replicas: 1
  placement:
    constraints:
      - node.role == worker

docker stack deploy -c vscode.yml vscode

```



Accessible sur : <http://192.168.56.11:8080> — mot de passe : `vscode123`

8.7 Récapitulatif des services

Service	URL d'accès	Réplicas	Nœud
Registry	http://192.168.56.10:5000	1/1	Manager
MariaDB	(interne)	1/1	Worker
Nginx	http://192.168.56.10	2/2	Workers
PHP	(interne)	2/2	Workers
VSCode Server	http://192.168.56.11:8080	1/1	Worker

```

laplateforme@swarm-manager:~/stacks$ docker node ls && echo "---" && docker service ls
ID                                HOSTNAME        STATUS    AVAILABILITY    MANAGER STATUS    ENGINE    VERSION
7jz128rb59ylxvzmuzao54p9w *     swarm-manager   Ready     Active           Leader             29.4.2
a05ee633fkd1zopqdkz0fgm83       swarm-worker1   Ready     Active           29.4.2
zqjfok95ed5gpxub6oygsl8r7       swarm-worker2   Ready     Active           29.4.2
z8t8m83rl1yc7npkjac09wx0yq       swarm-worker3   Ready     Active           29.4.2
---
ID            NAME                MODE         REPLICAS    IMAGE              PORTS
cx1wyfzzqnxr mariadb_mariadb     replicated   1/1          mariadb:10.11
28ok7v5226lg registry_registry   replicated   1/1          registry:2        *:5000->5000/tcp
ktbe7yqe6rkg vscode_vscode        replicated   1/1          codercom/code-server:latest *:8080->8080/tcp
spbrkoohowen web_nginx            replicated   2/2          nginx:alpine       *:80->80/tcp
2vxj0y9c2rx4 web_php              replicated   2/2          php:8.2-fpm

```

9. Tests PCA / PRA

Les tests PCA/PRA (Plan de Continuité d'Activité / Plan de Reprise d'Activité) permettent de vérifier que l'infrastructure est résiliente face aux pannes. Trois scénarios ont été simulés.

💡 *Le RTO (Recovery Time Objective) est le temps mesuré entre la panne et le retour à la normale. Plus il est court, meilleure est la résilience.*

9.1 Simulation de perte d'un conteneur

Ce test simule la panne brutale d'un conteneur Nginx. On tue manuellement le conteneur et on mesure le temps que met Swarm à le redémarrer automatiquement.

Procédure

```

# 1. Noter l'heure et identifier le conteneur
date && docker service ps web_nginx

# 2. Sur le worker concerné, tuer le conteneur
docker ps | grep nginx
docker rm -f <ID_CONTENEUR>

# 3. Sur le manager, surveiller la reprise
watch docker service ps web_nginx

```

Résultat observé

Swarm a détecté la panne (erreur code 137 = kill forcé) et a redémarré automatiquement le conteneur sur le même worker sans aucune intervention manuelle.

Paramètre	Valeur
Heure de la panne	12:53:19
Heure de reprise	12:54:37
RTO mesuré	~45 secondes
Erreur détectée	task: non-zero exit (137)
Action de Swarm	Redémarrage automatique sur le même worker
Résultat	✅ Service rétabli automatiquement


```

laplateforme@swarm-manager: ~
individual files in /usr/share/doc/*/*copyright.
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Tue May 5 09:53:42 2026 from 192.168.199.1
laplateforme@swarm-manager:~$ cd stacks
laplateforme@swarm-manager:~/stacks$ nano vscode.yml
laplateforme@swarm-manager:~/stacks$ docker stack deploy -c vscode.yml vscode
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network vscode_default
Creating service vscode_vscode
laplateforme@swarm-manager:~/stacks$ docker service ls
ID NAME IMAGE REPLICAS
c1wlyfz2qnkr mariadb mariadb replicated 1/1
28ok7v5226lg registry registry replicated 1/1
k1be7yge6fkg vscode vscode replicated 0/1
sbrkoo0hwen web nginx replicated 2/2
2xjy8y9c2rx4 web php replicated 2/2
laplateforme@swarm-manager:~/stacks$ docker service ps vscode_vscode --no-trunc
ID NAME IMAGE REPLICAS DESIRED STATE CURRENT STATE ERROR PORTS
lhao72ad8sdhdkkq340no vscode_vscode.1 codercom/code-server:latest@sha256:e6702766518b961c7e3c41095c129c7c9a82b041578c53326c52fa363a057727 swarm-worker3 Running Preparing 48 seconds ago
laplateforme@swarm-manager:~/stacks$ docker service ls
ID NAME IMAGE REPLICAS
c1wlyfz2qnkr mariadb mariadb replicated 1/1
28ok7v5226lg registry registry replicated 1/1
k1be7yge6fkg vscode vscode replicated 1/1
sbrkoo0hwen web nginx replicated 2/2
2xjy8y9c2rx4 web php replicated 2/2
laplateforme@swarm-manager:~/stacks$ docker node ls && echo "---" && docker service ls
ID HOSTNAME STATUS AVAILABILITY MANAGER STATUS ENGINE VERSION
7j128rb59ylxvzmza054p9w * swarm-manager Ready Active Leader 29.4.2
w05e033f41z0p0qkz0f9m3 swarm-worker1 Ready Active 29.4.2
2qj16k84e0dgp0ub0ygl1r7 swarm-worker2 Ready Active 29.4.2
z8t8m83ryc7npkjac09w0yq swarm-worker3 Ready Active 29.4.2
laplateforme@swarm-manager:~/stacks$ cd
laplateforme@swarm-manager:~$ date && docker service ps web nginx
mar. 05 mai 2026 12:50:45 CEST
ID NAME IMAGE REPLICAS DESIRED STATE CURRENT STATE ERROR PORTS
p0yvw76qfidy web nginx.1 nginx:alpine swarm-worker1 Running Running 2 hours ago
w7c73locj022 \ web nginx.1 nginx:alpine swarm-worker3 Shutdown Shutdown 2 hours ago
qfyo2nsqm2ac \ web nginx.1 nginx:alpine swarm-worker3 Shutdown Shutdown 2 hours ago
nrr1auqcz44 \ web nginx.1 nginx:alpine swarm-worker1 Shutdown Rejected 3 hours ago
tr1vb0uip8d8k1 \ web nginx.1 nginx:alpine swarm-worker3 Shutdown Rejected 20 hours ago
k5r792h7rr1l \ web nginx.2 nginx:alpine swarm-worker2 Running Running 2 hours ago
o171fn5kh30 \ web nginx.2 nginx:alpine swarm-worker1 Shutdown Shutdown 2 hours ago
o2f8frol97u \ web nginx.2 nginx:alpine swarm-worker1 Shutdown Shutdown 2 hours ago
a6eskcbskuwe \ web nginx.2 nginx:alpine swarm-worker2 Shutdown Rejected 3 hours ago
xv8m4hj1ex85 \ web nginx.2 nginx:alpine swarm-worker2 Shutdown Failed 3 hours ago
laplateforme@swarm-manager:~$

```

```

laplateforme@swarm-manager: ~
Toutes les 2,0s: docker service ps web nginx
ID NAME IMAGE REPLICAS DESIRED STATE CURRENT STATE ERROR PORTS
p0yvw76qfidy web nginx.1 nginx:alpine swarm-worker1 Running Running about a minute ago
w7c73locj022 \ web nginx.1 nginx:alpine swarm-worker3 Shutdown Failed about a minute ago "task: non-zero exit (137)"
qfyo2nsqm2ac \ web nginx.1 nginx:alpine swarm-worker3 Shutdown Shutdown 2 hours ago
nrr1auqcz44 \ web nginx.1 nginx:alpine swarm-worker1 Shutdown Rejected 3 hours ago
tr1vb0uip8d8k1 \ web nginx.1 nginx:alpine swarm-worker3 Shutdown Rejected 20 hours ago
k5r792h7rr1l \ web nginx.2 nginx:alpine swarm-worker2 Running Running 2 hours ago
o171fn5kh30 \ web nginx.2 nginx:alpine swarm-worker1 Shutdown Shutdown 2 hours ago
o2f8frol97u \ web nginx.2 nginx:alpine swarm-worker1 Shutdown Shutdown 2 hours ago
a6eskcbskuwe \ web nginx.2 nginx:alpine swarm-worker2 Shutdown Rejected 3 hours ago
xv8m4hj1ex85 \ web nginx.2 nginx:alpine swarm-worker2 Shutdown Failed 3 hours ago
laplateforme@swarm-manager:~$

```

9.2 Simulation de perte d'un nœud Worker

Ce test simule la panne complète d'une machine virtuelle worker. On éteint swarm-worker1 et on observe comment Swarm redistribue les conteneurs sur les workers restants.

Procédure

```
# 1. Éteindre swarm-worker1
sudo poweroff # depuis swarm-worker1
```

```
# 2. Sur le manager, surveiller le cluster
watch docker node ls

# 3. Surveiller la redistribution des services
watch docker service ps web_nginx
```

Résultat observé

Swarm a détecté la disparition de swarm-worker1 et a automatiquement replanifié les conteneurs sur swarm-worker3. Après redémarrage de worker1, le nœud a réintégré le cluster automatiquement avec le statut Ready.

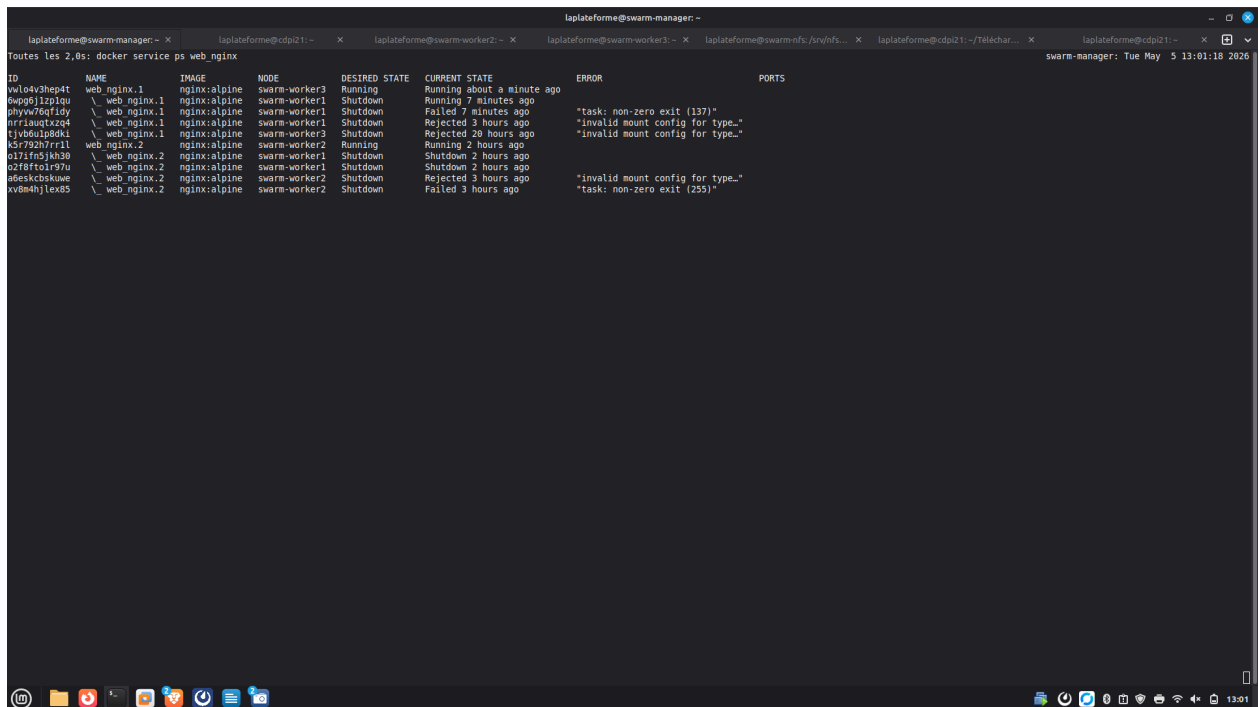
Paramètre	Valeur
Nœud en panne	swarm-worker1
Conteneurs affectés	web_nginx.1
Worker de basculement	swarm-worker3
RTO mesuré	~2 minutes
Réintégration du nœud	Automatique au redémarrage
Résultat	Basculement automatique réussi

💡 Le RTO est plus long qu'au test 1 car Swarm attend un timeout avant de considérer un nœud comme définitivement perdu, pour éviter les faux positifs.

Docker Swarm — Documentation Technique

```
laplateforme@swarm-manager: ~  
laplateforme@swarm-manager:~$ docker service ps vscode_vscode --no-trunc  
ID NAME IMAGE  
lha027ad8sdh2kdzku034dno vscode_vscode.1 codercom/code-server:latest@sha256:e6702766518b961c7e3c41895c129c7c9a82b041578c53326c52fa363a057727  
laplateforme@swarm-manager:~$ docker service ls  
ID NAME MODE REPLICAS IMAGE PORTS  
c2kbfzfzqnxr mariadb mariadb replicated 1/1 mariadb:10.11  
28okf5v526lg registry registry replicated 1/1 registry:2 *:5000->5000/tcp  
ktbe7yqe6rkq vscode_vscode replicated 1/1 codercom/code-server:latest *:8080->8080/tcp  
qabrkoohwen web_nginx replicated 2/2 nginx:alpine *:80->80/tcp  
2vxj9yc2rx4 web_php replicated 2/2 php:8.2-fpm  
laplateforme@swarm-manager:~$ docker node ls && echo "---" && docker service ls  
ID HOSTNAME STATUS AVAILABILITY MANAGER STATUS ENGINE VERSION  
71z128rb59ylvzmuzao54p9w * swarm-manager Ready Active Leader 29.4.2  
a05ee633fkdlzopgdz0fgm83 swarm-worker1 Ready Active 29.4.2  
zqjfk9d5d9pupb0ygs1sr7 swarm-worker2 Ready Active 29.4.2  
zdtbms3jy7npkjcc0bnxyq swarm-worker3 Ready Active 29.4.2  
...  
ID NAME MODE REPLICAS IMAGE PORTS  
c2kbfzfzqnxr mariadb mariadb replicated 1/1 mariadb:10.11  
28okf5v526lg registry registry replicated 1/1 registry:2 *:5000->5000/tcp  
ktbe7yqe6rkq vscode_vscode replicated 1/1 codercom/code-server:latest *:8080->8080/tcp  
qabrkoohwen web_nginx replicated 2/2 nginx:alpine *:80->80/tcp  
2vxj9yc2rx4 web_php replicated 2/2 php:8.2-fpm  
laplateforme@swarm-manager:~$ cd  
laplateforme@swarm-manager:~$ date && docker service ps web_nginx  
mar_05_mai_2026 12:50:45 CEST  
ID NAME IMAGE NODE DESIRED STATE CURRENT STATE ERROR PORTS  
pnyv76qfidy web_nginx.1 nginx:alpine swarm-worker1 Running Running 2 hours ago  
c2kbfzfzqnxr web_nginx.1 nginx:alpine swarm-worker3 Shutdown Shutdown 2 hours ago  
qfy02sqm2ac web_nginx.1 nginx:alpine swarm-worker3 Shutdown Shutdown 2 hours ago  
nrr1auqtxzq4 web_nginx.1 nginx:alpine swarm-worker1 Shutdown Rejected 3 hours ago "invalid mount config for type..."  
tlyb0ulp8dki web_nginx.1 nginx:alpine swarm-worker3 Shutdown Rejected 20 hours ago "invalid mount config for type..."  
k5r792h7rrll web_nginx.2 nginx:alpine swarm-worker2 Running Running 2 hours ago  
o17ifn5kh30 web_nginx.2 nginx:alpine swarm-worker1 Shutdown Shutdown 2 hours ago  
o2f8ftolr97u web_nginx.2 nginx:alpine swarm-worker1 Shutdown Shutdown 2 hours ago  
a8eskbckuue web_nginx.2 nginx:alpine swarm-worker2 Shutdown Rejected 3 hours ago "invalid mount config for type..."  
xv8mhjlex85 web_nginx.2 nginx:alpine swarm-worker2 Shutdown Failed 3 hours ago "task: non-zero exit (255)"  
laplateforme@swarm-manager:~$ watch docker service ps web_nginx  
laplateforme@swarm-manager:~$ date && docker service ps web_nginx  
mar_05_mai_2026 12:58:21 CEST  
ID NAME IMAGE NODE DESIRED STATE CURRENT STATE ERROR PORTS  
pnyv76qfidy web_nginx.1 nginx:alpine swarm-worker1 Running Running 4 minutes ago  
c2kbfzfzqnxr web_nginx.1 nginx:alpine swarm-worker1 Failed 5 minutes ago "task: non-zero exit (137)"  
qfy02sqm2ac web_nginx.1 nginx:alpine swarm-worker3 Shutdown Shutdown 2 hours ago  
nrr1auqtxzq4 web_nginx.1 nginx:alpine swarm-worker1 Shutdown Rejected 3 hours ago "invalid mount config for type..."  
tlyb0ulp8dki web_nginx.1 nginx:alpine swarm-worker3 Shutdown Rejected 20 hours ago "invalid mount config for type..."  
k5r792h7rrll web_nginx.2 nginx:alpine swarm-worker2 Running Running 2 hours ago  
o17ifn5kh30 web_nginx.2 nginx:alpine swarm-worker1 Shutdown Shutdown 2 hours ago  
o2f8ftolr97u web_nginx.2 nginx:alpine swarm-worker1 Shutdown Shutdown 2 hours ago  
a8eskbckuue web_nginx.2 nginx:alpine swarm-worker2 Shutdown Rejected 3 hours ago "invalid mount config for type..."  
xv8mhjlex85 web_nginx.2 nginx:alpine swarm-worker2 Shutdown Failed 3 hours ago "task: non-zero exit (255)"  
laplateforme@swarm-manager:~$
```

```
laplateforme@swarm-manager: ~  
laplateforme@swarm-manager:~$ docker service ps web_nginx  
Toutes les 2,0s: docker service ps web_nginx  
ID NAME IMAGE NODE DESIRED STATE CURRENT STATE ERROR PORTS  
vvl04v3hepat web_nginx.1 nginx:alpine swarm-worker3 Running Running about a minute ago  
pnyv76qfidy web_nginx.1 nginx:alpine swarm-worker1 Running Running 9 minutes ago  
qfy02sqm2ac web_nginx.1 nginx:alpine swarm-worker3 Shutdown Failed 8 minutes ago "task: non-zero exit (137)"  
nrr1auqtxzq4 web_nginx.1 nginx:alpine swarm-worker1 Shutdown Rejected 3 hours ago "invalid mount config for type..."  
tlyb0ulp8dki web_nginx.1 nginx:alpine swarm-worker3 Shutdown Rejected 20 hours ago "invalid mount config for type..."  
k5r792h7rrll web_nginx.2 nginx:alpine swarm-worker2 Running Running 2 hours ago  
o17ifn5kh30 web_nginx.2 nginx:alpine swarm-worker1 Shutdown Shutdown 2 hours ago  
o2f8ftolr97u web_nginx.2 nginx:alpine swarm-worker1 Shutdown Shutdown 2 hours ago  
a8eskbckuue web_nginx.2 nginx:alpine swarm-worker2 Shutdown Rejected 3 hours ago "invalid mount config for type..."  
xv8mhjlex85 web_nginx.2 nginx:alpine swarm-worker2 Shutdown Failed 3 hours ago "task: non-zero exit (255)"  
laplateforme@swarm-manager:~$
```



9.3 Test 3 — Persistance des données NFS

Ce test vérifie que les données stockées dans MariaDB survivent à la panne et au redémarrage du conteneur. C'est la validation du rôle du NFS dans l'architecture.

Procédure

```
# 1. Se connecter à MariaDB et créer des données de test
docker exec -it <ID_CONTENEUR> mariadb -u swarmuser -pswarmpassword swarmdb

CREATE TABLE test_pra (id INT, message VARCHAR(50));
INSERT INTO test_pra VALUES (1, 'donnee persistante');
SELECT * FROM test_pra;
EXIT;

# 2. Tuer le conteneur MariaDB
docker rm -f <ID_CONTENEUR>

# 3. Attendre le redémarrage automatique (~45 sec)
watch docker service ps mariadb_mariadb

# 4. Se reconnecter et vérifier les données
docker exec -it <NOUVEAU_ID> mariadb -u swarmuser -pswarmpassword swarmdb
SELECT * FROM test_pra;
```

Résultat observé

Après redémarrage du conteneur, la requête SELECT a retourné exactement les mêmes données qu'avant la panne. Le NFS garantit la persistance totale des données indépendamment du cycle de vie des conteneurs.

```
laplateforme@swarm-manager: ~
laplateforme@swarm-worker1: ~
laplateforme@swarm-worker2: ~
laplateforme@swarm-worker3: ~
laplateforme@swarm-nfs:/srv/nfs...
laplateforme@cdp21:~/Téléchar...
laplateforme@cdp21:~

laplateforme@cdp21:~$ ssh laplateforme@192.168.199.130
laplateforme@192.168.199.130's password:
Linux swarm-worker2 6.12.85-deb13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.85-1 (2026-04-30) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Mon May 4 14:13:30 2026 from 192.168.199.1
laplateforme@swarm-worker2:~$ ls /mnt/nfs/swarm/
ls: cannot access '/mnt/nfs/swarm/': No such file or directory
laplateforme@swarm-worker2:~$ docker ps | grep mariadb
29fdddc2556e   mariadb:10.11   "docker-entrypoint.s..." 15 minutes ago Up 15 minutes 3306/tcp mariadb_mariadb.1.wkq5xll1a753307d0cop9b5hz
laplateforme@swarm-worker2:~$ docker exec -it 29fdddc2556e mariadb -u swarmuser -p$P0W0RDSW0RDM0N1
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 3
Server version: 10.11.16-MariaDB-ubu2204 mariadb.org binary distribution

Copyright (c) 2009, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [swarmdb]> CREATE TABLE test_pra (id INT, message VARCHAR(50));
Query OK, 0 rows affected (0.053 sec)

MariaDB [swarmdb]> INSERT INTO test_pra VALUES (1, 'donnee persistante');
Query OK, 1 row affected (0.035 sec)

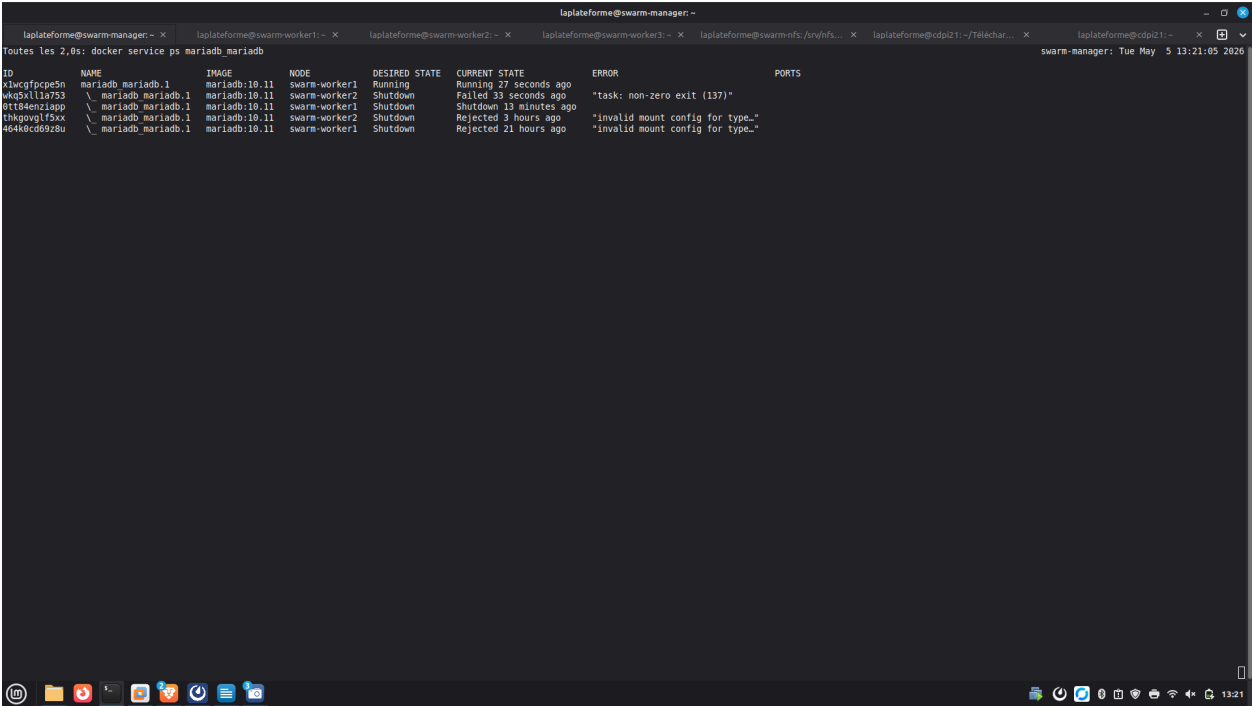
MariaDB [swarmdb]> SELECT * FROM test_pra;
+-----+-----+
| id | message |
+-----+-----+
| 1 | donnee persistante |
+-----+-----+
1 row in set (0.001 sec)

MariaDB [swarmdb]> EXIT;
```

Paramètre	Valeur
Données insérées	1 ligne : 'donnee persistante'
Conteneur tué	mariadb_mariadb.1
Données après reprise	✅ Intactes
Mécanisme	Stockage NFS persistant
Résultat	✅ Persistance des données validée

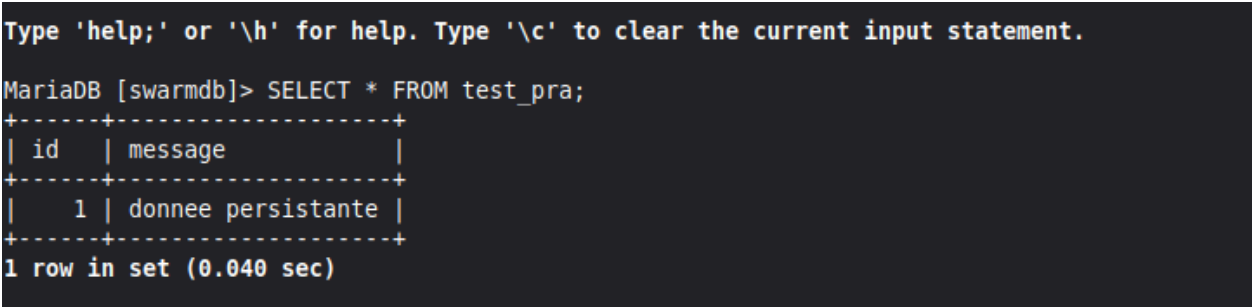
```
laplateforme@swarm-manager: ~
laplateforme@swarm-worker1: ~
laplateforme@swarm-worker2: ~
laplateforme@swarm-worker3: ~
laplateforme@swarm-nfs:/srv/nfs...
laplateforme@cdp21:~/Téléchar...
laplateforme@cdp21:~

Toutes les 2,0s: docker service ps mariadb_mariadb
ID            NAME                IMAGE                NODE                DESIRED STATE    CURRENT STATE                ERROR                PORTS
xlwcfpcpe5n   mariadb_mariadb.1   mariadb:10.11       swarm-worker1       Running          Running 3 seconds ago
wkq5xll1a753   \_ mariadb_mariadb.1   mariadb:10.11       swarm-worker2       Shutdown        Failed 19 seconds ago      "task: non-zero exit (137)"
p1t8end1app    \_ mariadb_mariadb.1   mariadb:10.11       swarm-worker1       Shutdown        Shutdown 12 minutes ago
thkgvgvlf5sx   \_ mariadb_mariadb.1   mariadb:10.11       swarm-worker2       Shutdown        Rejected 3 hours ago      "invalid mount config for type_"
464k0cd69z8u   \_ mariadb_mariadb.1   mariadb:10.11       swarm-worker1       Shutdown        Rejected 21 hours ago     "invalid mount config for type_"
```



The screenshot shows a terminal window titled 'laplateforme@swarm-manager: -'. The terminal displays the output of the command 'docker service ps mariadb mariadb'. The output is a table with columns: ID, NAME, IMAGE, NODE, DESIRED STATE, CURRENT STATE, ERROR, and PORTS. The table lists four containers. The first container, 'v1acpfpcpe5n', is in a 'Running' state. The other three containers are in a 'Shutdown' state with various error messages.

ID	NAME	IMAGE	NODE	DESIRED STATE	CURRENT STATE	ERROR	PORTS
v1acpfpcpe5n	mariadb_mariadb.1	mariadb:10.11	swarm-worker1	Running	Running 27 seconds ago		
wkq5xll1a753	_ mariadb_mariadb.1	mariadb:10.11	swarm-worker2	Shutdown	Failed 33 seconds ago	"task: non-zero exit (137)"	
9tt84enziapp	_ mariadb_mariadb.1	mariadb:10.11	swarm-worker1	Shutdown	Shutdown 13 minutes ago		
thkgvgvl15xx	_ mariadb_mariadb.1	mariadb:10.11	swarm-worker2	Shutdown	Rejected 3 hours ago	"invalid mount config for type_"	
464k8cd6928u	_ mariadb_mariadb.1	mariadb:10.11	swarm-worker1	Shutdown	Rejected 21 hours ago	"invalid mount config for type_"	



The screenshot shows a terminal window with a MariaDB prompt. The user has entered the command 'SELECT * FROM test_pra;'. The output shows a single row with the ID '1' and the message 'donnee persistante'.

```
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
MariaDB [swarmdb]> SELECT * FROM test_pra;
+-----+-----+
| id  | message                |
+-----+-----+
| 1   | donnee persistante    |
+-----+-----+
1 row in set (0.040 sec)
```

9.4 Récapitulatif des tests

Test	Scénario	RTO	Résultat
Test 1	Perte d'un conteneur Nginx	~45 secondes	Redémarrage auto
Test 2	Perte d'un nœud Worker	~2 minutes	Basculement auto
Test 3	Persistence données NFS	-	Données intactes